3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 20 – Constructors & Methods

FOUNDATION

Creation of Class Program

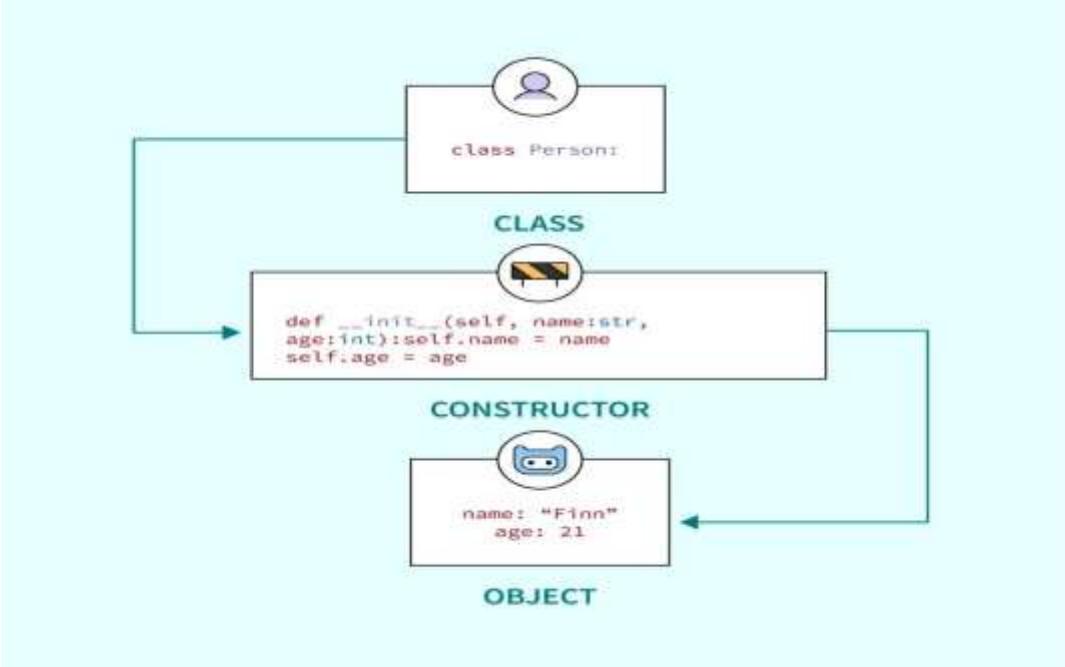
```
#To define the Student calss and create an Object to it.

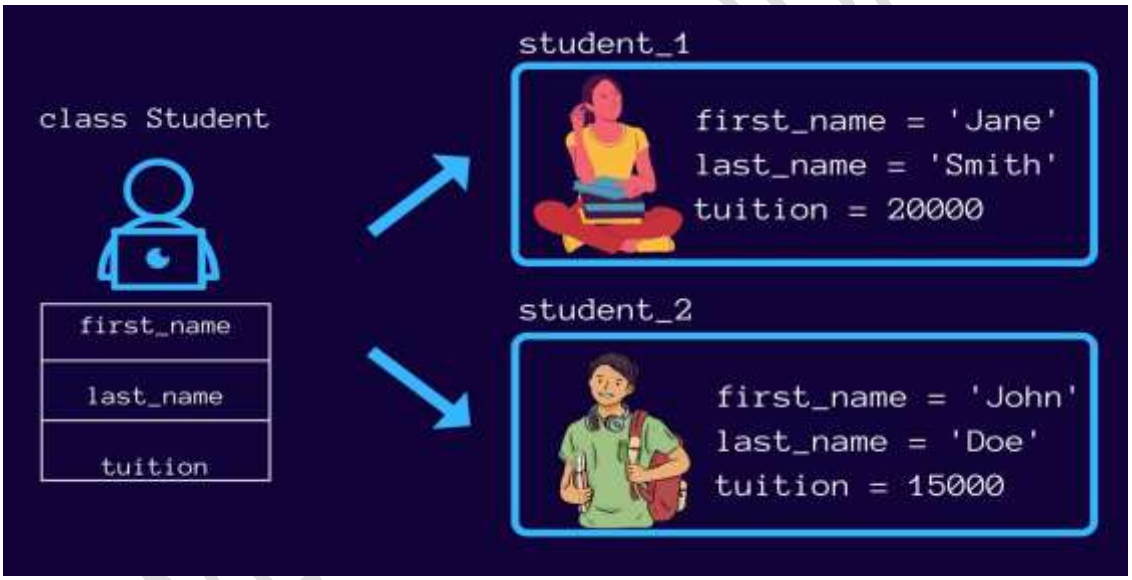
#Class Definition
class Student:
    #Special method called constructor
    def __init__(self):
        self.name = "Ram"
        self.age = 21
        self.marks = 75.90

    #This is an instance method
    def putdata(self):
        print("Name: ", self.name)
        print("Age: ", self.age)
        print("Marks: ", self.marks)

#Create an instance to the student class
s = Student()

#Call the method using an Object
s.putdata()
```





1. Learning Objectives

By the end of this class, students will be able to:

- Understand **constructors** in Python.
- Understand **methods** inside a class.
- Use the __init__() constructor.
- Understand the purpose of self.

- Create a Student class.
- Create multiple student objects.
- Store student information using attributes.
- Create methods to display and calculate student information.
- Build a small **Student Management Program**.

2. What is a Constructor?

A **constructor** is a special method that is automatically called when an object is created.

In Python, the most commonly used constructor is:

__init__()

Simple idea

When we create a student object:

s1 = Student("Rahul", 101, 85)

Python automatically calls:

__init__()

to initialize the object's data.

Real-world example

Think about filling an admission form.

When a new student joins a school, we may enter:

- Student Name
- Roll Number
- Marks

3DWEBSOFT FOUNDATION

The constructor can automatically store this information when the student object is created.

3. Constructor Syntax

```
class Student:


   def __init__(self, name, roll_no, marks):
      self.name = name
      self.roll_no = roll_no
      self.marks = marks
```

Creating an object:

```
s1 = Student("Rahul", 101, 85)
```

Here:

Part	Meaning
Student	Class name
s1	Object
__init__()	Constructor
name	Student name
roll_no	Roll number
marks	Marks
self	Current object

3DWEBSOFT FOUNDATION

4. Understanding self

self refers to the **current object**.

For example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

When we create:

```
s1 = Student("Asha", 90)
```

Python stores:

s1.name → "Asha"

s1.marks → 90

When we create:

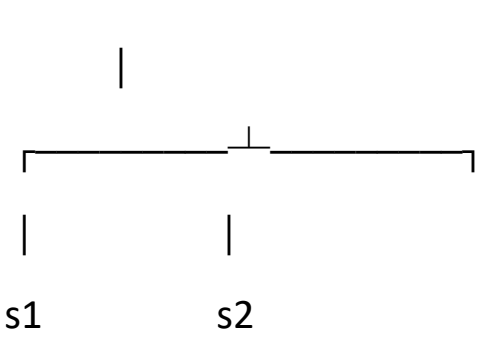
```
s2 = Student("Ravi", 75)
```

Python stores:

s2.name → "Ravi"

s2.marks → 75

Visual Concept



SUWEBSOFT FOUNDATION

| |

Asha, 90 Ravi, 75

Each object has its **own data**.

5. What is a Method?

A **method** is a function defined inside a class.

Methods describe what an object can **do**.

For example, a Student object can:

- Display details
- Calculate grade
- Check pass/fail
- Calculate percentage

Example:

```
class Student:
    def display(self):
        print("Student details")
```

Calling the method:

```
s1.display()
```

6. Constructor vs Method

SUWEBSOFT FOUNDATION

Constructor	Method
Special method	Normal function inside a class
Usually __init__()	Can have any meaningful name
Automatically called when object is created	Usually called explicitly
Initializes object data	Performs an action
Example: __init__()	Example: display()

7. Student Class – Basic Example

```
class Student:

    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("Marks:", self.marks)
```

3DWEBSOFT FOUNDATION

```
s1 = Student("Rahul", 101, 85)

s1.display()
```

Output

```
Name: Rahul

Roll No: 101

Marks: 85
```

8. How the Program Works

Create Student Class

↓

Define __init__()

↓

Create Student Object

↓

Pass student information

↓

Constructor initializes data

↓

Call display()

↓

Student information displayed

9. Constructor with Default Values

3DWEBSOFT FOUNDATION

A constructor can also have default values.

```
class Student:

    def __init__(self, name="Unknown", marks=0):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)
s1 = Student()
s1.display()
```

Output

```
Name: Unknown
Marks: 0
```

10. Multiple Objects

One class can create many objects.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
```

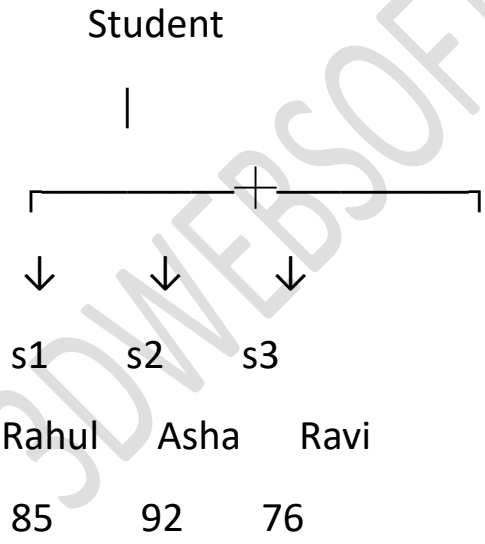
3DWEBSOFT FOUNDATION

```
        print(self.name, self.marks)
s1 = Student("Rahul", 85)
s2 = Student("Asha", 92)
s3 = Student("Ravi", 76)
s1.display()
s2.display()
s3.display()
```

Output

```
Rahul 85
Asha 92
Ravi 76
```

Important Concept



11. Creating a Method to Check Pass/Fail

```
class Student:
```

3DWEBSOFT FOUNDATION

```
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def check_result(self):
        if self.marks >= 35:
            print(self.name, "has Passed")
        else:
            print(self.name, "has Failed")
s1 = Student("Rahul", 75)
s2 = Student("Ravi", 25)
s1.check_result()
s2.check_result()
```

Output

```
Rahul has Passed
Ravi has Failed
```

12. Student Grade Method

We can create a method to calculate the grade.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def grade(self):
```

3DWEBSOFT FOUNDATION

```
        if self.marks >= 90:
            return "A+"
        elif self.marks >= 80:
            return "A"
        elif self.marks >= 70:
            return "B"
        elif self.marks >= 60:
            return "C"
        elif self.marks >= 35:
            return "D"
        else:
            return "F"
s1 = Student("Asha", 92)
print("Name:", s1.name)
print("Grade:", s1.grade())
```

Output

```
Name: Asha
Grade: A+
```

13. Student Percentage Example

Suppose a student has marks in five subjects.

```
class Student:
    def __init__(self, name, marks):
```

3DWEBSOFT FOUNDATION

```
        self.name = name
        self.marks = marks
    def total(self):
        return sum(self.marks)
    def percentage(self):
        return self.total() / len(self.marks)
s1 = Student("Rahul", [80, 75, 90, 85, 70])
print("Name:", s1.name)
print("Total:", s1.total())
print("Percentage:", s1.percentage())
```

Output

```
Name: Rahul
Total: 400
Percentage: 80.0
```

14. Complete Student Class

Now let's combine constructor and multiple methods.

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks
```

3DWEBSOFT FOUNDATION

```
def display(self):
    print("\nStudent Details")
    print("Name:", self.name)
    print("Roll No:", self.roll_no)
    print("Marks:", self.marks)
def check_result(self):
    if self.marks >= 35:
        return "Pass"
    else:
        return "Fail"
def grade(self):
    if self.marks >= 90:
        return "A+"
    elif self.marks >= 80:
        return "A"
    elif self.marks >= 70:
        return "B"
    elif self.marks >= 60:
        return "C"
    elif self.marks >= 35:
        return "D"
    else:
        return "F"
```

3DWEBSOFT FOUNDATION

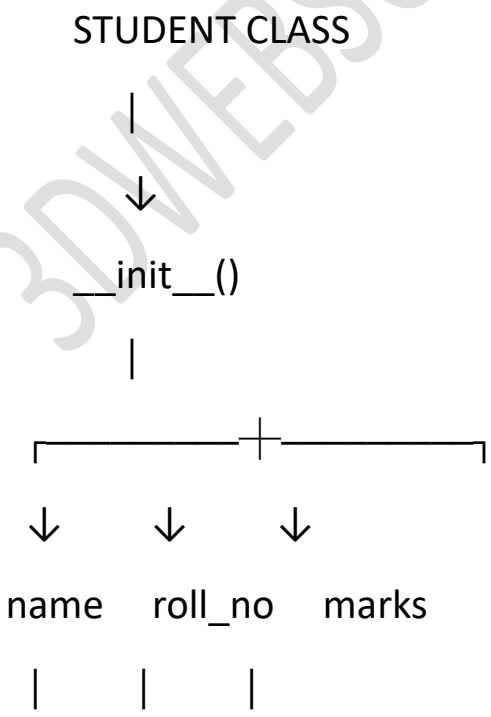
```
s1 = Student("Rahul", 101, 85)
s1.display()
print("Result:", s1.check_result())
print("Grade:", s1.grade())
```

Output

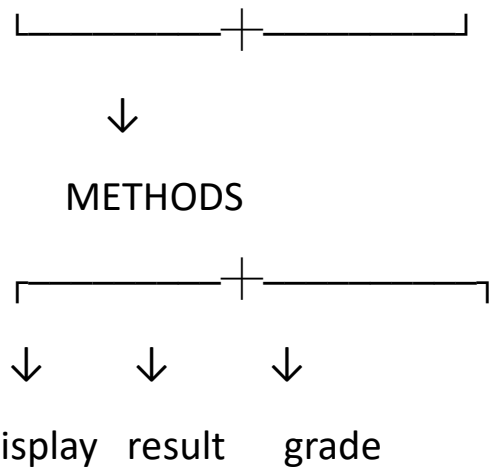
```
Student Details
Name: Rahul
Roll No: 101
Marks: 85
Result: Pass
Grade: A
```

15. Constructor → Attributes → Methods

The complete relationship can be understood as:



3DWEBSOFT FOUNDATION



16. Constructor Execution

Consider:

s1 = Student("Asha", 101, 92)

Internally, Python calls approximately:

Student.__init__(s1, "Asha", 101, 92)

Therefore:

self = s1

So:

self.name = name

becomes:

s1.name = "Asha"

And:

self.marks = marks

becomes:

s1.marks = 92

This is an important concept for students to understand.

3DWEBSOFT FOUNDATION

17. Method with Parameters

Methods can accept additional parameters.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def add_bonus(self, bonus):
        self.marks += bonus
    def display(self):
        print(self.name, self.marks)
s1 = Student("Rahul", 80)
s1.add_bonus(5)
s1.display()
```

Output

```
Rahul 85
```

18. Method Returning a Value

Methods don't always have to print something.

They can **return** a value.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

3DWEBSOFT FOUNDATION

```
    def get_grade(self):
        if self.marks >= 90:
            return "A+"
        elif self.marks >= 80:
            return "A"
        elif self.marks >= 70:
            return "B"
        else:
            return "C"
s1 = Student("Asha", 88)
grade = s1.get_grade()
print("Grade:", grade)
```

Output

Grade: A

19. Practical Program – Student Management

Objective

Create a simple Student application using:

- Constructor
- Attributes
- Methods
- Objects
- Conditional statements

3DWEBSOFT FOUNDATION

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks
    def display(self):
        print("\n--- Student Details ---")
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("Marks:", self.marks)
    def result(self):
        if self.marks >= 35:
            print("Result: PASS")
        else:
            print("Result: FAIL")
    def grade(self):
        if self.marks >= 90:
            print("Grade: A+")
        elif self.marks >= 80:
            print("Grade: A")
        elif self.marks >= 70:
            print("Grade: B")
        elif self.marks >= 60:
```

3DWEBSOFT FOUNDATION

```
            print("Grade: C")
        elif self.marks >= 35:
            print("Grade: D")
        else:
            print("Grade: F")
name = input("Enter student name: ")
roll_no = int(input("Enter roll number: "))
marks = float(input("Enter marks: "))
student = Student(name, roll_no, marks)
student.display()
student.result()
student.grade()
```

Sample Output

```
Enter student name: Fathima
Enter roll number: 101
Enter marks: 87
--- Student Details ---
Name: John
Roll No: 101
Marks: 87.0
Result: PASS
Grade: A
```

3JWEBSOFT FOUNDATION

20. Practical Activity – Create an Employee Class

Students can now apply the same concept to another real-world problem.

Requirements

Create an Employee class with:

- Employee name
- Employee ID
- Salary

Create methods:

- display()
- annual_salary()
- salary_grade()

Starter code:

```
class Employee:
    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
        self.salary = salary
    def display(self):
        print("Name:", self.name)
        print("Employee ID:", self.emp_id)
        print("Salary:", self.salary)
```

3DWEBSOFT FOUNDATION

```
    def annual_salary(self):
        return self.salary * 12
e1 = Employee("Rahul", 501, 30000)
e1.display()
print("Annual Salary:", e1.annual_salary())
```

21. Real-World Applications

Constructors and methods are used in many applications.

Application	**Possible Class**
School Management	Student
Banking	BankAccount
Hospital	Patient
E-commerce	Product
Transport	Car
Employee Management	Employee
Library	Book
College	Teacher
Food Delivery	Order

For example:

BankAccount

|

3DWEBSOFT FOUNDATION

```
    ├── account_number
    ├── holder_name
    └── balance
        |
        ├── deposit()
        ├── withdraw()
        └── display_balance()
```

22. Common Mistakes

Mistake 1: Forgetting self

Wrong:

```
class Student:


   def __init__(name, marks):
      self.name = name
      self.marks = marks
```

Correct:

```
class Student:
   def __init__(self, name, marks):
      self.name = name
      self.marks = marks
```

3DWEBSOFT FOUNDATION

Mistake 2: Forgetting double underscores

Wrong:

def init(self, name):

Correct:

def __init__(self, name):

Mistake 3: Calling method incorrectly

Wrong:

student.display

Correct:

student.display()

The parentheses () call the method.

Mistake 4: Incorrect attribute name

If we define:

self.roll_no = roll_no

we should use:

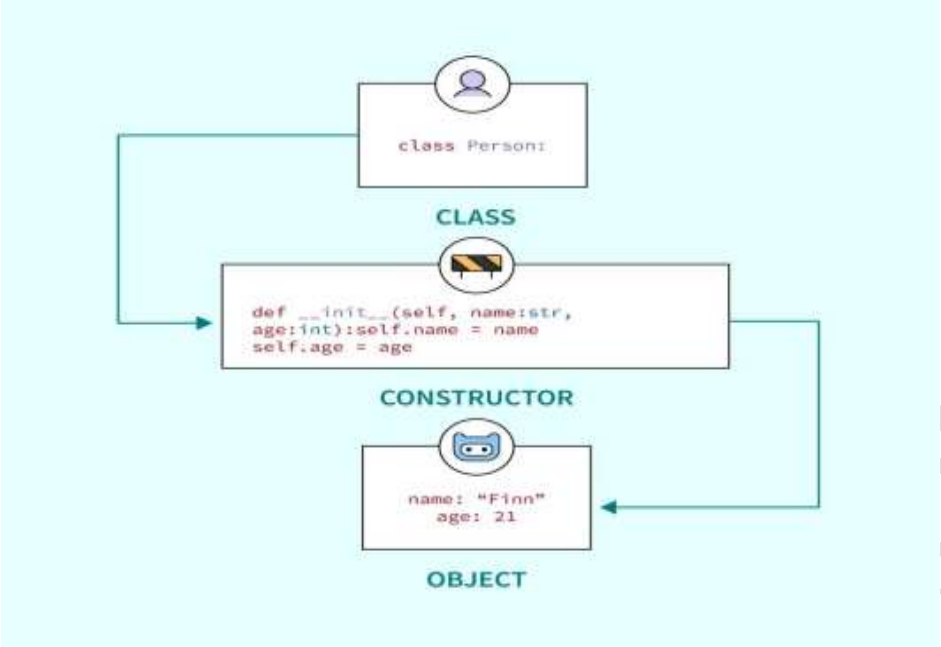
print(self.roll_no)

not:

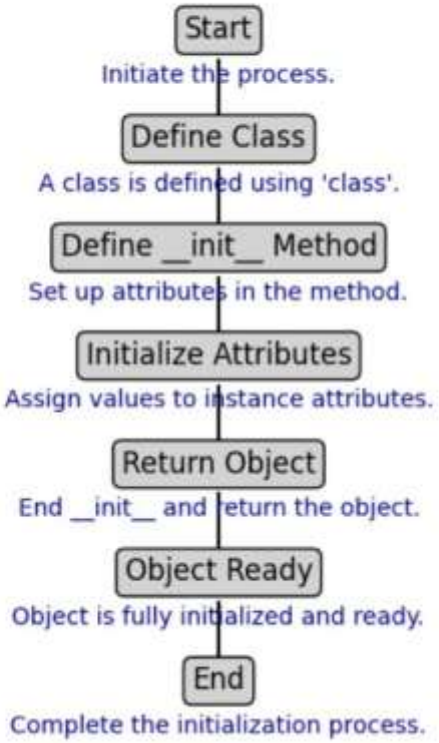
print(self.roll)

23. Constructor and Method Flow

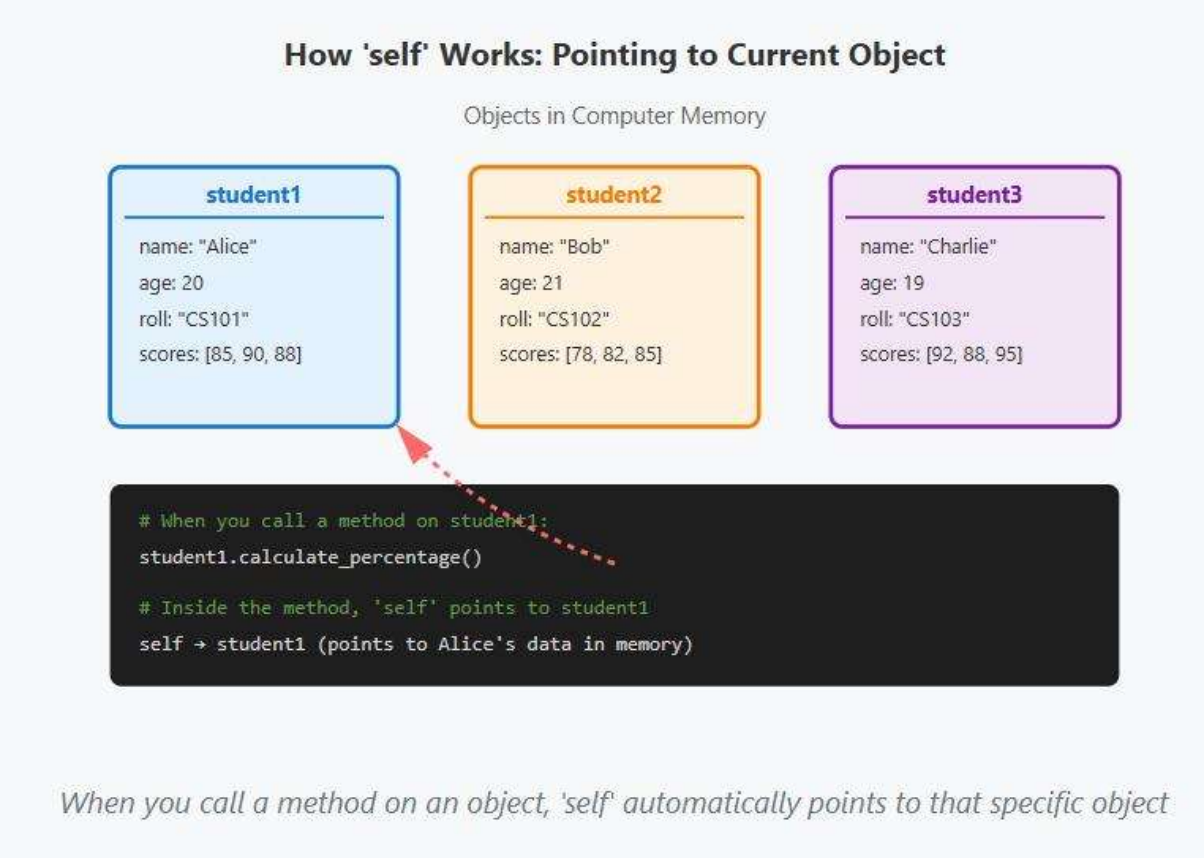
JUWEBSOFT FOUNDATION



Concept of the __init__ Method in Python



https://emitechlogic.com/



START

↓

Create Student Class

↓

Define Constructor

↓

Create Object

↓

__init__() executes automatically

↓

Store Student Data

↓

3DWEBSOFT FO

Call Methods

↓

Display / Result / Grade

↓

END

24. Mini Project – Student Report Card

Project Requirements

Create a Student class that stores:

- Name
- Roll number
- English marks
- Maths marks
- Science marks

Create methods:

1. display_details()
2. total_marks()
3. percentage()
4. result()
5. grade()

Suggested Program

class Student:

3DWEBSOFT FOUNDATION

```
def __init__(self, name, roll_no, english, maths, science):
    self.name = name
    self.roll_no = roll_no
    self.english = english
    self.maths = maths
    self.science = science
def display_details(self):
    print("\n--- Student Report ---")
    print("Name:", self.name)
    print("Roll No:", self.roll_no)
    print("English:", self.english)
    print("Maths:", self.maths)
    print("Science:", self.science)
def total_marks(self):
    return self.english + self.maths + self.science
def percentage(self):
    return self.total_marks() / 3
def result(self):
    if self.english >= 35 and self.maths >= 35 and self.science >= 35:
        return "PASS"
    return "FAIL"
def grade(self):
    percentage = self.percentage()
```

3DWEBSOFT FOUNDATION

```
        if percentage >= 90:
            return "A+"
        elif percentage >= 80:
            return "A"
        elif percentage >= 70:
            return "B"
        elif percentage >= 60:
            return "C"
        elif percentage >= 35:
            return "D"
        else:
            return "F"
student = Student("Asha", 101, 90, 85, 88)
student.display_details()
print("Total:", student.total_marks())
print("Percentage:", student.percentage())
print("Result:", student.result())
print("Grade:", student.grade())
```

Expected Output

```
--- Student Report ---
Name: Asha
Roll No: 101
English: 90
```

3DWEBSOFT FOUNDATION

Maths: 85

Science: 88

Total: 263

Percentage: 87.66666666666667

Result: PASS

Grade: A

25. Classroom Practical Activities

Activity 1 – Student

Create a Student class with:

name

roll_no

marks

Methods:

display()

result()

grade()

Activity 2 – Book

Create:

class Book:

Attributes:

3DWEBSOFT FOUNDATION

- title
- author
- price

Methods:

- display()
- discount()

Activity 3 – Bank Account

Create:

class BankAccount:

Attributes:

- account holder
- account number
- balance

Methods:

- deposit()
- withdraw()
- display_balance()

3DWEBSOFT FOUNDATION

26. Quick Revision

Constructor

__init__()

Initializes object data.

Attribute

Stores information.

self.name

self.marks

Method

Performs an action.

def display(self):

Object

An instance of a class.

s1 = Student(...)

self

Refers to the current object.

27. Important Questions

Q1. What is a constructor?

A constructor is a special method used to initialize an object's attributes when the object is created.

Q2. What is the constructor name in Python?

__init__()

Q3. What is a method?

A method is a function defined inside a class.

Q4. Why is self used?

3DWEBSOFT FOUNDATION

self refers to the current object and allows us to access its attributes and methods.

Q5. Is __init__() called manually?

Normally, no. Python automatically calls it when an object is created.

28. MCQs

1. Which method is commonly used as a constructor?

A. start()
B. main()
C. __init__()
D. create()

Answer: C

2. What does self represent?

A. Class name
B. Current object
C. Python interpreter
D. Module

Answer: B

3. Which of these is a method?

A. name
B. marks
C. display()
D. Student

Answer: C

4. Which statement creates an object?

3DWEBSOFT FOUNDATION

s1 = Student("Asha", 90)

A. Class creation
B. Object creation
C. Method creation
D. Module creation

Answer: B

5. Which symbol is required when calling a method?

student.display___

A. []
B. {}
C. ()
D. <>

Answer: C

29. Practical Assignment

Create a **Student Management Class** with the following:

Class: Student

Attributes:

1. name
2. roll_no
3. marks

Methods:

3DWEBSOFT FOUNDATION

1. display()

2. check_result()

3. calculate_grade()

4. update_marks()

Challenge

Create **5 student objects** and display their:

- Name
- Roll number
- Marks
- Result
- Grade

Then identify:

- Highest marks
- Lowest marks
- Class average

30. Concept Map

OOP

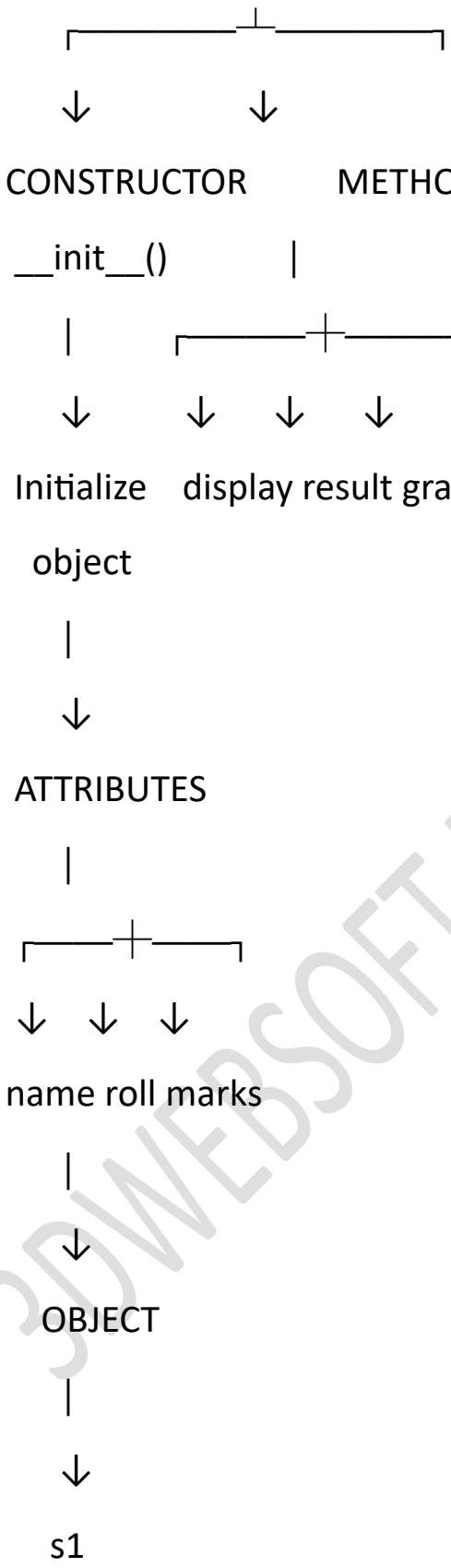
|

↓

CLASS

|

3DWEBSOFT FOUNDATION



3DWEBSOFT FOUNDATION

Final Takeaway

Students should remember these four key ideas:

CLASS → Blueprint

OBJECT → Real instance

CONSTRUCTOR → Initializes data

METHOD → Performs an action

For a Student class:

```
class Student:


    def __init__(self, name, marks):
        self.name = name
        self.marks = marks


    def display(self):
        print(self.name, self.marks)
```

Creating and using the object:

```
s1 = Student("Asha", 90)
s1.display()
```

Class → Object → Constructor → Attributes → Methods is the core concept students should master in this lesson.

3DWEBSOFT FOUNDATION